MOBL-08: Session Replay for Mobile

> **Series:** MOBL — Mobile Monitoring | **Notebook:** 8 of 12 | **Created:** February 2026 | **Last Updated:** 10/29/2026

Overview

Mobile Session Replay gives a video-like reconstruction of user sessions for debugging and UX analysis. This notebook covers enabling Session Replay (a server-side setting), configuring privacy masking in code, choosing a capture percentage to control costs, leveraging crash replay, and querying session data with DQL.

Table of Contents

1. What is Mobile Session Replay?
2. Enabling Session Replay
3. Privacy Masking Levels
4. Sampling & Cost Control
5. Crash Session Replay
6. Platform Configuration
7. Querying Session Data

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail
Mobile SDK	OneAgent for Android 8.303+ / OneAgent for iOS 8.323+ (Dynatrace version 1.303+)
Session Replay License	DEM units with Session Replay entitlement
Permissions	`storage:user.sessions:read`, `storage:user.events:read` (replay playback additionally needs `storage:user.replays:read`)

Requirement	Details
Platform	iOS 15+ (Swift 5+, Xcode 16+) or Android 6.0+ (API 23+); native apps; Flutter and React Native as a preview on Latest Dynatrace with DPS licensing; not available for Cordova, Xamarin or .NET MAUI
Data	At least 24 hours of mobile user action data

1. What is Mobile Session Replay?

Mobile Session Replay lets you replay each tap, swipe and screen rotation of a user session "in a movie-like experience". In Dynatrace's own words it is *"a video-like reconstruction of the user interactions with mobile applications that use captured events and data"*; on iOS, the `DTXDebugMasking` environment variable (Xcode scheme > Run > Arguments) shows the screenshots Session Replay takes, which is how you check masking during development.

Two capture modes exist:

1. **Full Session Replay** -- a configurable percentage of sessions is captured.
2. **Session Replay on crashes** -- every session that ends in a crash is captured, regardless of the Full Session Replay setting.

Key Use Cases

Use Case	Description
Bug reproduction	Watch exactly what the user did before encountering an error, eliminating guesswork
UX friction analysis	Identify confusing navigation patterns, rage taps, and abandoned flows
Crash context	View the session leading up to a crash to understand the sequence of events
Conversion optimization	Trace drop-off points in checkout or onboarding funnels
Support escalation	Attach session replays to support tickets for faster resolution

> **Note:** Session Replay Classic is available only for native iOS and Android apps. On Latest Dynatrace with DPS licensing, Session Replay for **Flutter** and **React Native** is available as a preview (pages published 09/30/2026 and 09/28/2026; register for the preview program; not available for RUM Classic or Dynatrace Managed). The older Session Replay overview still says it *"is not supported on Cordova, React Native, Flutter, Xamarin, and .NET MAUI"* — the newer preview pages are the current statement. Cordova, Xamarin and .NET MAUI are not supported; for hybrid apps, only the native part is replayed.
>
> **Sources:** Session Replay for Flutter (DT docs) — *"Session Replay for Flutter is available as a preview."*, *"The preview is available to customers with Latest Dynatrace and DPS licensing."*; Session Replay for React Native (DT docs) —

> *"Session Replay for React Native is available as a preview."*
>
> **Correction (09/28/2026).** Earlier revisions said Session Replay "is **not** a video recording", captures "UI state rather than pixels", and typically adds "< 50 KB per session". None of this is supported by the Session Replay documentation, which describes screenshots and a video-like reconstruction.
>
> **Sources:** Session Replay Classic for iOS (DT docs) — *"Session Replay is a video-like reconstruction of the user interactions with mobile applications that use captured events and data."*, Session Replay Classic for Android (DT docs) — *"Session Replay is not available for cross-platform frameworks such as Cordova, React Native, Flutter, Xamarin, and more"*

2. Enabling Session Replay

Session Replay is switched on **in the mobile app's settings in Dynatrace**, not in `Info.plist` or the Gradle block. Once the app is instrumented (instrumentation wizard complete), the steps are the same for iOS and Android. The path below is the one the *Session Replay Classic* pages document; on a New RUM Experience frontend, look for the same toggles under the frontend's **Settings > Enablement and cost control**:

1. Go to **Mobile** and select the mobile application.
2. Select **More (…) > Edit** in the upper-right corner of the application tile.
3. From the application settings, select **General > Enablement and cost control**.
4. Turn on **Enable Full Session Replay** and/or **Enable Session Replay on crashes**.

Setting	Effect
Full Session Replay at 100%	All sessions are captured
Full Session Replay below 100%	A random selection of sessions is captured -- this percentage is the only sampling control
Session Replay on crashes	All sessions with a crash are captured, regardless of the Full Session Replay setting and percentage

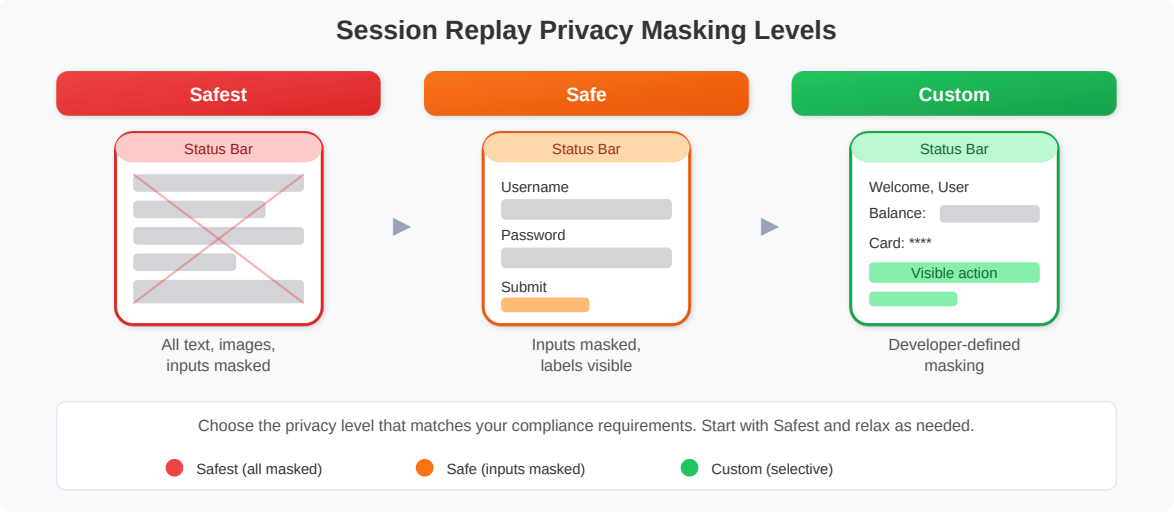
Then complete the Session Replay steps of the instrumentation wizard for your platform. There is **no client-side sample rate**, and no `Info.plist` or Gradle key that sets a sample rate or a privacy mode; masking is configured in code (Section 3). One client-side key does exist: for an instrumented **SwiftUI** app, also set `DTXSwiftUIEnableSessionReplayInstrumentation` to `true` in `Info.plist` (*"Set the DTXSwiftUIEnableSessionReplayInstrumentation configuration key to true in your project's Info.plist file."* — Session Replay for iOS).

> **Correction (09/28/2026).** Earlier revisions configured Session Replay with Info.plist keys (`DTXSessionReplayEnabled` , `DTXSessionReplayPrivacyMode` , `DTXSessionReplaySampleRate` , `DTXSessionReplayOnCrash`) and Gradle properties (`sessionReplay` , `sessionReplayPrivacyMode` ,

> `sessionReplaySampleRate`, `sessionReplayOnCrash`), and described an SDK sample rate that combines with the server rate. None of these exists on the Session Replay pages.
>
> **Sources:** Session Replay Classic for Android (DT docs) — *"From the application settings, select General > Enablement and cost control . Turn on Enable Full Session Replay or Enable Session Replay on crashes ."*, Session Replay Classic for iOS (DT docs).

3. Privacy Masking Levels

Privacy is a critical concern when capturing session replays. Dynatrace provides three masking levels to protect sensitive user data.



Level	Description	Masked Elements
Safest	Maximum privacy	Editable text fields, images, labels, web views and switches masked
Safe	Balanced approach	Editable text fields masked; labels, images, web views and switches visible
Custom	Developer-defined	By default the same as Safest; mask/unmask specific views via the API

OneAgent applies **Safest** by default. To use Safe or Custom, set the masking level through the API.

Choosing the Right Level

- **Safest** (default) -- Use for apps handling financial, healthcare, or highly regulated data. Replays show layout and navigation but no readable content.
- **Safe** -- Only editable text fields are masked; labels and navigation elements remain visible for context.
- **Custom** -- Starts from Safest; developers then mask or unmask specific views for precise control.

Setting the Masking Level

iOS (Swift):

```
let maskingConfiguration = MaskingConfiguration(maskingLevelType: .safe)
try? AgentManager.setMaskingConfiguration(maskingConfiguration)
```

Android (Java):

```
MaskingConfiguration config = new MaskingConfiguration.Safe(); // .Safest or .Custom
DynatraceSessionReplay.setConfiguration(Configuration.builder()
    .withMaskingConfiguration(config)
    .build());
```

Custom Masking Examples

iOS -- mask views by `accessibilityIdentifier` (Custom level):

```
try? maskingConfiguration.addMaskedView(viewIds: ["masked_view_id"])
try? maskingConfiguration.addNonMaskedView(viewIds: ["nonMasked_view_id"])
```

Android -- mask views by ID (Custom level), then apply the configuration as above:

```
Set&lt;Integer&gt; set = new HashSet&lt;Integer&gt;() { add(R.id.view_id1);
add(R.id.view_id2); };
new MaskingConfiguration.Custom().addMaskedIds(set);
```

Android Jetpack Compose -- mask a composable:

```
import com.dynatrace.agent.compose.api.dtMask

@Composable
fun MyScreen() {
    Column {
        Text(text = "This text will be masked", modifier = Modifier.dtMask())
    }
}
```

On both platforms, a view whose `accessibilityIdentifier` (iOS) or `android:tag` (Android) contains the masking tag `data-dtrum-mask` is always masked.

> **Correction (09/28/2026).** Earlier revisions used a `dtxMaskingMode` view property (not documented) and labelled an `applyUserPrivacyOptions` call "mask a specific view" -- that call sets the data-collection level (MOBL-09), not Session Replay masking.
>
> **Tip:** Start with the default **Safest** level or with **Safe**, and move to **Custom** only when specific elements need to be unmasked for debugging. This keeps privacy by default.
>
> **Sources:** Session Replay Classic for iOS (DT docs) — *"Safest —all the editable text fields, images, labels, web views, and switches are masked. Safe —all the editable text fields are masked."*, *"Custom —by default, masks the same elements as Safest"*, Session Replay Classic for Android (DT docs).

4. Sampling & Cost Control

Session Replay is billed on top of the session itself — in **DEM (Digital Experience Monitoring) units** on classic licensing; on a Dynatrace Platform Subscription, check the Session Replay line of your rate card. The Full Session Replay percentage under **General > Enablement and cost control** (Section 2) is the primary lever for managing costs while still capturing enough data for meaningful analysis.

Sampling Strategy

The Full Session Replay percentage determines what share of user sessions is recorded for replay. Not every session needs to be captured -- statistical sampling provides representative coverage.

In community practice, teams start from percentages like these and tune them to their traffic and budget:

Environment	Starting Percentage	Rationale
Production	5--10%	Captures enough sessions for trend analysis while controlling DEM consumption
Staging / QA	100%	Full coverage for pre-release testing and bug verification
High-traffic production	1--5%	Even 1% of millions of sessions provides thousands of replays
Post-incident	Temporarily increase to 50--100%	Raise sampling when actively investigating a reported issue

Cost Estimation

Session Replay cost depends on:

1. **Number of captured sessions** -- Directly proportional to sample rate and total traffic.
2. **Session length** -- Longer sessions generate more replay data.
3. **UI activity** -- in community practice, busier screens (frequent transitions and animation) produce more replay data per session.

Formula:

```
Monthly replay sessions = Monthly active sessions x Sample rate (%)
DEM units consumed = Monthly replay sessions x Avg DEM units per session
```

Best Practices

- **Start low** -- Begin with 5% in production and increase only if you need more coverage.
- **Use crash replay** -- Even at low sample rates, crash replay captures the sessions that matter most (see next section).
- **Monitor consumption** -- Track DEM unit usage in the Dynatrace license overview to avoid surprises.

- **Segment by app** -- Set a different Full Session Replay percentage per mobile application based on its criticality.

5. Crash Session Replay

Crash Session Replay is one of the most valuable features of mobile Session Replay. It works independently of the general sampling rate to ensure that crash sessions are always captured.

How It Works

With **Enable Session Replay on crashes** turned on, every session that ends in a crash is captured, whatever the Full Session Replay percentage. The replay shows the user actions that preceded the crash.

Key Benefits

Benefit	Description
Always-on crash capture	Crash replays are saved regardless of the general sample rate
Full context	See the exact sequence of screens and interactions leading to the crash
Faster root cause	Developers can visually reproduce the crash scenario without relying on user descriptions

Configuration

Crash replay is the **Enable Session Replay on crashes** toggle in the app settings (**General > Enablement and cost control**, Section 2). There is no `Info.plist` or Gradle key for it. Separately, the user's privacy choice must allow it: the OneAgent SDK privacy options carry a `crashReplayOptedIn` flag (MOBL-09 §3).

> **Sources:** Session Replay Classic for Android (DT docs) — *"Enabling Session Replay on Crashes means guarantees that, regardless of the Enable Full Session Replay setting and its const and traffic control value, all sessions with crash are will be captured."* (sic)

6. Platform Configuration

Enablement and the capture percentage live in the app settings (Section 2), and masking is set in code (Section 3). The one platform key is iOS-only: SwiftUI apps also need `DTXSwiftUIEnableSessionReplayInstrumentation` set to `true` in `Info.plist` (Section 2). What differs by platform is the requirements:

	iOS	Android
Agent	OneAgent for iOS 8.323+	OneAgent for Android 8.303+
OS	iOS 15.0+ (not tvOS or iPadOS)	Android 6.0+ (API 23+)
Toolchain	Swift 5+, Xcode 16+; SwiftUI supported	Android Gradle plugin 8.1.1+, Kotlin 2.1.0+; Jetpack Compose 1.4+ from OneAgent 8.325
Masking API	`MaskingConfiguration` + `AgentManager.setMaskingConfiguration`	`MaskingConfiguration` + `DynatraceSessionReplay.setConfiguration`; `Modifier.dtMask()` for Compose
Debug check	`DTXDebugMasking` environment variable (Xcode scheme) shows the screenshots taken	Session Replay logs as for OneAgent

Cross-Platform Frameworks

Framework	Session Replay Support	Notes
React Native	Preview (Latest Dynatrace + DPS)	`@dynatrace/react-native-plugin` 2.349.1+ with `@dynatrace/react-native-session-replay`; register for the preview program
Flutter	Preview (Latest Dynatrace + DPS)	`dynatrace_flutter_plugin` 3.349.1+ with `dynatrace_flutter_plugin_replay`; register for the preview program
Xamarin / .NET MAUI	Not available	Both named as unsupported on the Session Replay overview
Cordova / Ionic	Not available	For hybrid apps, only the native part is replayed

7. Querying Session Data

Use DQL to analyze mobile session patterns, identify high-activity sessions, track trends, and find crash sessions for replay review.

Session Counts by Mobile Application

Count distinct sessions (`dt.rum.session.id` on `user.events`) per mobile application over the last 24 hours to understand session volume distribution.

```
// Session counts by mobile application
fetch user.events, from:-24h
| filter dt.rum.application.type == "mobile"
| summarize session_count = countDistinct(dt.rum.session.id), by:{frontend.name}
| sort session_count desc
```

Most Active Sessions by Action Count

Identify the most active sessions in the last 24 hours from `user.sessions`, which carries per-session counters (`user_action_count`, `request_count`, `error.count`) and `characteristics.has_replay`. High action counts may indicate power users, automated testing, or potential abuse.

```
// Most active sessions by user-action count -- one user.sessions record per session
fetch user.sessions, from:-24h
| filter dt.rum.application.type == "mobile"
| fields start_time, dt.rum.session.id, user_action_count, request_count, error.count,
characteristics.has_replay
| sort user_action_count desc
| limit 20
```

Daily Session Volume Trends

Track session volume over the past 7 days, and how many sessions carry a replay (`characteristics.has_replay`), to identify usage patterns, weekend vs. weekday differences, and growth trends.

```
// Daily session volume trends, and how many carry a replay
fetch user.sessions, from:-7d
| filter dt.rum.application.type == "mobile"
| makeTimeseries {sessions = count(), replay_sessions = countIf(characteristics.has_replay
== true)}, time:start_time, interval:24h
```

Crash Sessions for Replay Review

Find recent crash sessions (`user.sessions` with `error.has_crash`) to review their Session Replay; `characteristics.has_replay` tells you whether a replay exists. These are the highest-priority sessions for debugging, and crash replay ensures they are captured even at low sampling rates.

```
// Crash sessions for replay review
fetch user.sessions, from:-24h
| filter dt.rum.application.type == "mobile" and error.has_crash == true
| fields start_time, dt.rum.session.id, os.name, app.short_version,
characteristics.has_replay
| sort start_time desc
| limit 20
```

Summary

This notebook covered the key aspects of Mobile Session Replay:

Topic	Key Takeaway
What is Session Replay	Video-like reconstruction from captured events and screenshots; native iOS/Android; Flutter / React Native in preview (Latest Dynatrace + DPS)
Enabling	App settings > General > Enablement and cost control; no client-side keys
Privacy masking	Three levels (Safest default, Safe, Custom), set in code via `MaskingConfiguration`
Capture percentage	Full Session Replay percentage; start at 5--10% for production, 100% for staging/QA
Crash replay	Captures every crashed session regardless of the Full Session Replay percentage
Platform requirements	iOS 15+ / OneAgent 8.323+; Android 6.0+ / OneAgent 8.303+
Querying	Use DQL on `user.sessions` / `user.events` to analyze session patterns and find crash sessions

Next Steps

Continue to **MOBL-09: Session Properties & Data Privacy** to explore session properties, user tagging, data-collection levels and opt-in, and data deletion.

References

- Session Replay for Mobile
- Session Replay Classic for Android (DT docs)
- Session Replay Classic for iOS (DT docs)
- Mobile SDK Privacy Settings
- Dynatrace Platform Subscription (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.